Asset Intelligence Copilot

System design and evaluation results | FlytBase Local Challenge, Hacktoberfest 2026 | Siddhant Pardeshi | github.com/siddhant-pardeshi/asset-intelligence-copilot

Chat UI Cards, tables, charts, map	**FastAPI router** Intent + memory + evidence	**Domain tools** Status, downtime, spills, SOP	**SQLite WAL** Fleet + documents + memory

1. How the system works

Ingestion and retrieval. The app merges the supplied fleet data into SQLite, preserving record IDs for traceable answers. Indexed well/date fields support time-window and per-well lookups. Manuals, SOPs, and technician reports are stored as searchable text; thresholds and some operational rules are derived from those documents. The configured data roots allow an additional dataset directory to be loaded without changing the query code.

Deterministic domain layer. Python tools compute thermal delta and producing status from inspection readings and well-type thresholds, count downtime as seven days per non-producing weekly inspection, filter corrected spill reports, and evaluate compliance facts. This keeps core domain calculations reproducible. An optional OpenAI-compatible LLM path can assist with broader queries or prose when configured; it is not required for the built-in scenarios.

Evidence and visual answers. The router selects a status card, table, temperature trend, downtime bar chart, spill map, or area-by-month heatmap. Responses carry evidence references to inspections, maintenance items, notes, and documents so operators can inspect the source.

Memory and conflicts. Shared field notes and corrections are available across users; private conversation references are scoped to the active user. Corrections preserve their source and scope (one well or well type). When operator statements conflict with telemetry or other notes, the UI surfaces the disagreement as unconfirmed instead of silently treating it as fact.

2. Engineering choices and limits

Why this design: domain arithmetic and state rules remain inspectable; flexible routing and document lookup cover mixed structured and free-text records; the same ingestion path can merge a new data root. The app runs locally with no API key in its default deterministic mode. Optional LLM behavior depends on the provider, key, and model configured by the operator.

Model and cost: no model is required for the built-in scenarios, so default operation has no model API usage or model charge. A user may configure an OpenAI-compatible model for broader questions or prose; actual model, latency, and cost depend on that configuration and were not measured for this submission.

Next: strengthen ingestion validation for new schemas, add retrieval-quality and adversarial-query checks, measure latency on the released test dataset, and add an audited vision pipeline only if drone imagery becomes available. The present project analyzes supplied inspection records; it does not perform image recognition.

Evaluation results

Recorded project evaluation: 63 of 63 assertion checks passed across the 18 supplied scenarios. Results below summarize the documented checks; the evaluator can reproduce them with scripts/run_eval.py.

Coverage	Verified outcome
Status and stale records	Thermal readings overrule stale recorded status; Q1 identifies WELL-0001 as not producing and cites the latest inspection.
Fleet and trend visuals	Area filtering returns the expected stopped wells; weekly temperature history renders as a line chart; downtime ranking and spill map/heatmap return the documented records.
Spills and compliance	Corrected false positives are excluded; spill totals, regulator thresholds, containment windows, and open notices are represented with source references.
Memory and correction	Private references resolve only within a user; shared notes reach other users; sensor/spill corrections affect later answers at the selected scope.
Conflict honesty	Contradictory operator claims and conflicting notes are presented with their sources and marked as unresolved where appropriate.

Selected scenario results

Q1 WELL-0001 status: thermal delta 3.2 C against 15 C T200 threshold; latest drone inspection and shut-in context cited.

Q2 / Q12b Northeast Block 8 stopped wells: four initially; after the WELL-0164 reading correction, three remain.

Q5 / Q6 Downtime follows weekly non-producing inspections: 21 days for WELL-0001 in 2025; top Block 7 wells are ranked with 7-day increments.

Q9 / Q10 Compliance checks surface documented breaches and two late-containment cases against the 4-hour limit.

Q11 / Q15 / Q16 Private memory, cross-user correction propagation, and shared workover announcement behavior are covered.

Q17 / Q18 A conflicting “running fine” claim is flagged against thermal evidence; conflicting spill notes are shown alongside repair evidence.

What worked and what remains

The strongest result is the combination of reproducible domain calculations with mixed-source evidence and user-scoped memory. The documented evaluation covers the supplied scenarios, not arbitrary future datasets. Broader natural-language coverage, new schema variants, real deployment latency, and live provider behavior need validation against the final test data and demo environment. No independent rerun is claimed in this write-up.

Run locally: install backend/requirements.txt, start the FastAPI app from backend, and open http://localhost:8000. To reproduce the recorded evaluation, run python scripts/run_eval.py from the repository root. MIT licensed.